MODULE 16 · DOMAIN 2 - IMPLEMENT GENERATIVE AI AND AGENTIC SOLUTIONS

Observability

Tracing with OpenTelemetry, spans and attributes, token analytics, latency breakdowns, safety signals and user feedback.

DOMAIN 2 - Generative AI and agents WEIGHT 30-35% MODULE 16 of 39

admin@sthithapragna.com

Monitoring says **something** is wrong. A trace is the **flight recorder** that shows where, and why.

Evaluation, monitoring and tracing

INSTRUMENT	ANALOGY	ANSWERS
Evaluation (module 15)	The MOT before the road	Is quality good enough to release?
Monitoring (module 08)	The dashboard while driving	Is something wrong right now?
Tracing (this module)	The flight recorder	What exactly happened in this run?

ON THE EXAM

Aggregate numbers find the problem. Individual traces explain it. Scenarios usually need both, in that order.

What a trace is made of

TERM	MEANING
Trace	The whole journey of one request through the system
Span	One operation in that journey, with a start and end time
Child span	Nested work, such as a tool call inside an agent run
Attribute	A key-value detail on a span: tokens, tool arguments, IDs
Exporter	Sends spans to a backend, such as Application Insights

ON THE EXAM

Foundry follows OpenTelemetry's GenAI semantic conventions, so traces read the same across frameworks.

Common GenAI spans and attributes

NAME	KIND	RECORDS
`invoke_agent`	Span	An agent being invoked; can contain other agents
`plan`	Child span	A planning or task decomposition phase
`invoke_workflow`	Span	A coordinated workflow of agents or steps
`execute_tool`	Child span	One tool execution
`gen_ai.tool.call.arguments`	Attribute	The arguments sent to a tool
`gen_ai.tool.call.result`	Attribute	What the tool returned

HONEST LIMITS

These conventions have Development status in OpenTelemetry, so names may still change.

Server-side or client-side tracing?

SERVER-SIDE

Automatic

- Captured for agents running in Foundry
- No code changes needed
- Covers the platform's side of the run

CLIENT-SIDE (PREVIEW)

Your application code

- Instrument your app with OpenTelemetry
- Covers your own logic and calls
- Export to App Insights, console, or OTLP tools

ON THE EXAM

Trace context propagation, on by default when tracing is enabled, joins both sides into one trace.

Switching tracing on

01
Connect
Link Application Insights to the project.

02
Roles
Log Analytics Reader to view traces.

03
Opt in
Set the tracing variable to true.

04
Instrument
Call instrument(), then configure Azure Monitor.

05
Run
Traces arrive in 2 to 5 minutes.

06
Inspect
Traces view: up to 90 days.

THE SIGNAL

'Traces don't appear' has three usual causes: no App Insights connected, a missing role, or tracing never opted in.

Client-side tracing to Application Insights

```
import os
os.environ["AZURE_EXPERIMENTAL_ENABLE_GENAI_TRACING"] = "true"

from azure.ai.projects import AIProjectClient
from azure.ai.projects.telemetry import AIProjectInstrumentor
from azure.monitor.opentelemetry import configure_azure_monitor

AIProjectInstrumentor().instrument()
project = AIProjectClient(endpoint=endpoint, credential=credential)
conn = project.telemetry.get_application_insights_connection_string()
configure_azure_monitor(connection_string=conn)
client = project.get_openai_client()
```

ON THE EXAM

The variable must be set before instrument() runs, and clients should be created after it.

Tracing environment variables

VARIABLE	DEFAULT	CONTROLS
AZURE_EXPERIMENTAL_ENABLE_GENAI_TRACING	false	Whether tracing happens at all
OTEL_INSTRUMENTATION_GENAI_CAPTURE_MESSAGE_CONTENT	false	Recording messages and tool arguments
..._ENABLE_TRACE_CONTEXT_PROPAGATION	true	Joining client and server spans
..._TRACE_CONTEXT_PROPAGATION_INCLUDE_BAGGAGE	false	Sending baggage, which can carry PII
AZURE_TRACING_GEN_AI_INCLUDE_BINARY_DATA	false	Images and file data in spans

ON THE EXAM

Parameters passed to instrument() override the matching environment variables.

Content recording off does **not** hide everything. The **@trace_function** decorator always records parameters and return values.

Keeping telemetry safe

- Enable content recording **in development**; keep it **off in production** unless compliance allows it.
- Never put **secrets, credentials or tokens** in prompts, tool arguments or span attributes.
- Redact personal data before it reaches telemetry, or route sensitive content to a restricted table.
- Treat traces as production data: **access control** and **retention policies**, as for logs.
- Application Insights charges by volume and retention, so tracing has a running cost.

> ON THE EXAM
>
> Traces often contain more sensitive data than the app's own database, because they record the conversation itself.

Where the tokens go

- Every response reports **input, cached, output and reasoning** tokens in its usage details.
- Traces record token consumption **per step**, so you can see which call spends most.
- Look for **repeated tool loops** and **growing history**: later steps cost more than early ones.
- Compare **cached** with input tokens to check prompt caching is actually working.
- The Agent Monitoring Dashboard shows token usage trends across agents.

> ON THE EXAM
>
> A cost spike on the dashboard is a question. Per-step tokens in a trace are the answer.

Where the time goes

- Each span has a **start and end time**, so a trace shows the duration of every step.
- Compare child spans: **model calls, tool executions, retrieval**, planning.
- For streaming, separate **time to first token** from total time, as module 08 showed.
- Pair latency with tokens: a slow step that generated many tokens may be behaving normally.
- A deployment-level latency metric can't show that a slow **tool** is the real problem.

THE TIEBREAKER

'The agent is slow' is diagnosed by span durations, not by upgrading or shrinking the model.

Where safety shows up in your telemetry

SIGNAL	WHERE YOU FIND IT
Guardrail results per response	content_filters in Responses API output (module 11)
Blocked prompts	HTTP 400 with the code content_filter
Blocked and harmful volume trends	Content Safety metrics in Azure Monitor (module 08)
Safety scores on live traffic	Continuous evaluation (module 08)
Resistance to attacks	Red teaming attack success rate (module 10)

ON THE EXAM

Add guardrail outcomes as span attributes, so a blocked request is visible inside the trace that explains it.

Attaching feedback to traces

- Capture **thumbs up or down**, or a **rating**, from users in your app.
- Send it as a **gen_ai.evaluation.result** OpenTelemetry event.
- Emit it in the **same trace context**, with the same trace and span ID, or it won't attach.
- Annotations are **appended**, never overwritten. Builders can also annotate in the portal.
- Every project has a default template: thumbs up scores 1.0 pass, thumbs down 0.0 fail.

> ON THE EXAM
>
> Feedback on the trace turns 'users are unhappy' into 'these twelve runs, and here is what they did'.

The debugging loop

01
Alert
A metric or feedback trend moves.

02
Find
Filter traces for affected runs.

03
Read
Spans, durations, tokens, arguments.

04
Fix
Prompt, tool, model or data.

05
Evaluate
Prove the fix on a dataset.

06
Release
Through the CI gate.

ON THE EXAM

Convert the troublesome traces into an evaluation dataset, so the same failure is tested on every future release.

A developer adds client-side tracing. Inside a startup function, they call AIProjectInstrumentor().instrument(), and a few lines later set AZURE_EXPERIMENTAL_ENABLE_GENAI_TRACING to true.

No spans appear anywhere, not even in a console exporter running locally.

What should the developer change?

A Grant themselves the Log Analytics Reader role on Application Insights.

B Set the message content capture variable to true as well.

C Wait up to two hours for traces to propagate to Azure Monitor.

D Set the tracing variable to true before calling instrument().

◆ PAUSE AND DECIDE

What should the developer change?

A. Log Analytics Reader.
Needed to view traces in Azure, but the local console shows none either.

B. Enable content capture.
Adds message content to spans. It doesn't create spans that don't exist.

C. Wait two hours.
Propagation takes 2 to 5 minutes, and a console exporter has no delay at all.

D. Opt in before instrumenting.
The variable is read when instrument() runs. Set afterwards, tracing never starts.

THE SIGNAL

'Not even in the console' rules out every Azure-side cause. The problem is in the application itself.

> A research agent takes around 14 seconds to answer. The team assumes the model is slow and is about to switch to a smaller model.
>
> Tracing is enabled, and traces for slow requests are available in the Foundry portal.

What should the team do first?

A Check Time to Last Byte for the model deployment in Azure Monitor.

B Raise the deployment's tokens-per-minute allocation to reduce waiting.

C Open slow traces and compare model call and tool span durations.

D Switch to the smaller model and compare user satisfaction a week later.

◆ PAUSE AND DECIDE

What should the team do first?

A Deployment Time to Last Byte.

Shows model response time only. It can't reveal a slow tool or search step.

B Raise TPM.

Only helps if requests are throttled, and nothing suggests they are.

C Compare span durations in traces.

Shows exactly which step takes the time before anything is changed.

D Switch model, check later.

Acts on an assumption, and may cut quality without touching the real cause.

THE TIEBREAKER

Measure before you change. A span breakdown often shows a slow tool, not a slow model.

> A healthcare agent runs in production. Compliance forbids patient details in telemetry. Content recording is off.
>
> To debug failures, an engineer decorated the lookup_patient(patient_name) function with @trace_function. Patient names now appear in traces.

What should the team do?

A. Remove the decorator; add a manual span with only safe attributes.

B. Turn content recording off again, which also stops the decorator recording.

C. Enable content recording only outside business hours, when volume is low.

D. Enable baggage propagation so patient data is sent separately from spans.

◆ PAUSE AND DECIDE

What should the team do?

A Manual span, safe attributes only.

Keeps duration and failure visibility while choosing exactly what gets recorded.

B Content recording off.

It's already off. The decorator records parameters regardless of that setting.

C Record outside business hours.

Patient data in telemetry is forbidden at any hour.

D Enable baggage propagation.

Baggage can carry PII to Azure services. It makes exposure worse.

THE SIGNAL

The decorator ignores the content setting. For sensitive functions, write the span yourself.

A chat app lets users give a thumbs down. The app sends each one as an OpenTelemetry event to Application Insights, from a separate background worker.

The events are stored, but none of them appear as feedback on the matching traces in the Foundry portal.

What should the developers change?

A Save the feedback in a separate database table, keyed by user ID.

B Emit the event with the interaction's trace and span IDs.

C Enable message content recording so feedback text is captured in spans.

D Record each thumbs down as a custom metric in Azure Monitor instead.

◆ PAUSE AND DECIDE

What should the developers change?

A. Separate database table.

Stores feedback, but further from the traces, not attached to them.

B. Same trace context.

Feedback attaches to a trace only when emitted with its trace and span ID.

C. Enable content recording.

Unrelated. Feedback correlation depends on trace context, not message content.

D. Custom metric.

Aggregates feedback, and loses the link to individual traces entirely.

THE SIGNAL

'Stored but not attached' is a correlation problem. A separate worker usually loses the trace context.

Observability in five lines

- **A trace is the flight recorder:** spans, durations, tokens and tool arguments, step by step.
- **Opt in before instrumenting.** No App Insights, no role, or no opt-in means no traces.
- **Content recording is off by default,** but @trace_function always records parameters.
- **Latency and cost:** compare spans before changing the model.
- **Feedback attaches** only in the same trace context.

> ON THE EXAM
>
> Next, module 17 opens the agent modules: roles, goals, instructions, conversation tracking and tool schemas.